国立大学法人
電気通信大学
The University of Electro-Communications

Pham Laboratory
Integrated circuit design laboratory

「Course」
EDHOC on a Rocket RV64 SoC

「Lesson 1」 EDHOC SoC on Arty A7-100T

Khai-Duy Nguyen

2026/12

Today's lecture purpose:

- Turn the EDHOC hardware requirement into a Chipyard SoC: 64-bit Rocket, 8 KiB caches, 64 KiB on-chip RAM, GPIO / UART / JTAG / SPI.
- Set up your PC and boards from a fresh clone, then map the SoC onto two Arty A7-100T boards: SD card, radio, UART and JTAG on the four PMOD ports.
- Build the bitstream, close timing at 75 MHz, program both boards and check every peripheral with a test program.

Repository: github.com/Khaiduy/chipyard-edhoc (private: ask the author for access) · paths are relative to its root

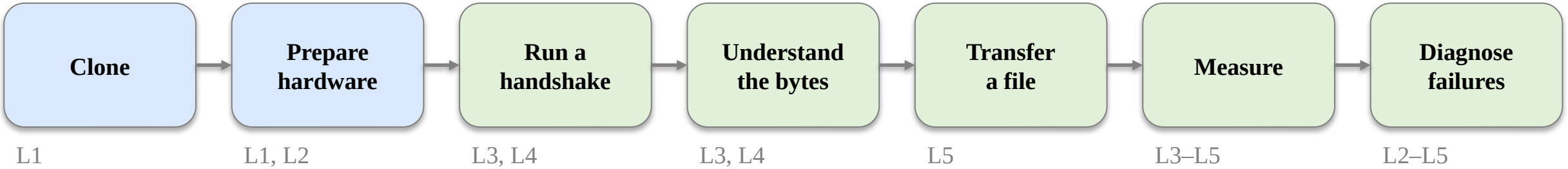
Outline

1. Start Here
2. The System You Will Build
3. SoC Configuration
4. FPGA Pins & Harness
5. Build & Timing Closure
6. Programming & Bring-up
7. Practice

Outline

1. **Start Here**
2. The System You Will Build
3. SoC Configuration
4. FPGA Pins & Harness
5. Build & Timing Closure
6. Programming & Bring-up
7. Practice

1. Start Here (1/8) The learning path



- At the end of the course you can, with only these slides and the repository:
  - configure **your own** two boards (cables, consoles, wiring);
  - run one classical (RFC 9528) and one post-quantum (AuthKEM) handshake and explain the important output lines;
  - send a file from one SD card to the other and prove it arrived intact;
  - find the cause of a deliberately broken build or link, and restore it.

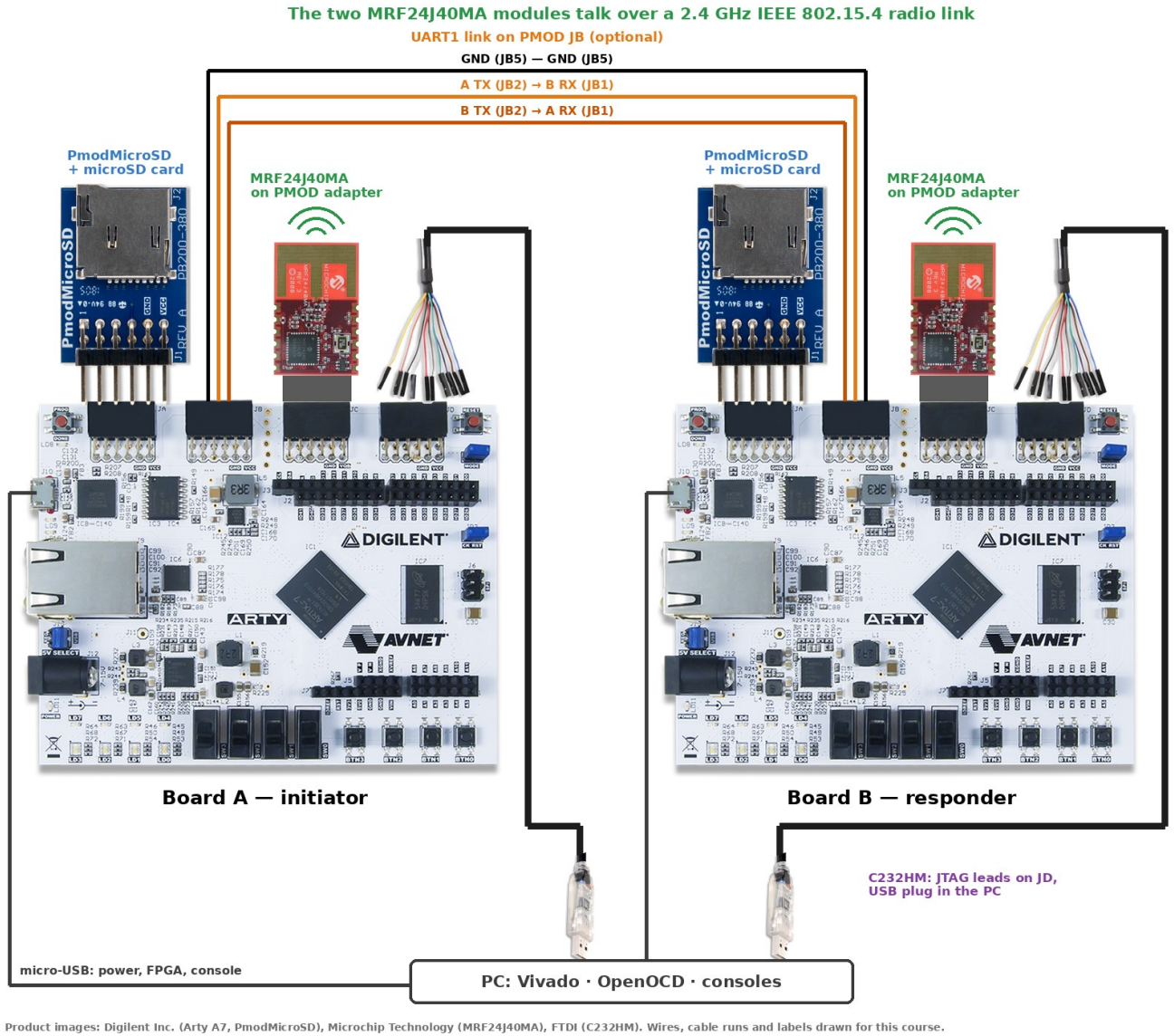
Lesson map: 1 hardware SoC · 2 software and first runs · 3 EDHOC · 4 post-quantum EDHOC · 5 file transfer, performance, diagnosis.

1. Start Here (2/8) Equipment

Item	Qty	Notes
Digilent Arty A7-100T	2	one board per party (initiator A, responder B)
micro-USB cable	2	power, FPGA programming and the console of each Arty
FTDI C232HM-DDHSL-0 USB–JTAG cable (3.3 V)	2	core debug on PMOD JD; leave its red VCC wire unconnected
Digilent PmodMicroSD + microSD card	2	PMOD JA; any SDHC / SDXC card — it gets erased
MRF24J40MA 802.15.4 module	2	PMOD JC; module-to-JC signal wiring in section 4 (6/9)
SD card reader (optional)	1	copy your own files; without one, sd_prep does it over JTAG
3 jumper wires (optional)	–	wired UART1 link on PMOD JB (GND, TX↔RX crossed); 3× faster than the radio

Wiring of every PMOD, pin by pin: section 4 (2/9).

1. Start Here (3/8) How everything is connected



Same wiring on both boards; the 3 UART wires on JB are optional (TRANSPORT=uart). Pin by pin: section 4 (2/9).

1. Start Here (4/8) Software

Software	Tested version	Needed for
Ubuntu (x86-64)	24.04 LTS	host PC
Vivado ML Standard (free) or Vivado Lab	2024.2	building (route A) and programming the FPGAs
Chipyard and its conda environment	1.11.0 + this fork	route A only
riscv64-unknown-elf-gcc, rv64imac / lp64 multilib	15.1.0	all firmware
OpenOCD with RISC-V and FTDI support	0.12.0+dev	loading firmware over JTAG
Python 3	3.11	host tests, timing tables, reference model
pip: pyfatfs pycryptodome cryptography kyber-py	–	FAT32 host test, AuthKEM reference model
dosfstools (mkfs.vfat, fsck.fat)	4.2	FAT32 host test, mksd.sh

- USB access without sudo: install Vivado's cable drivers (Digilent) and OpenOCD's *60-openocd.rules* (C232HM), and be in the *dialout* / *plugdev* groups for the consoles.
- Other versions may work; these are the ones every result in the course came from.

1. Start Here (5/8) Get the code — two routes

```
Run this — on your PC
git clone git@github.com:Khaiduy/chipyard-edhoc.git chipyard
cd chipyard
```

Route	Your goal	Setup	Then
A — full hardware build	learn how the SoC is generated and placed on the FPGA	./build-setup.sh riscv-tools; source env.sh; ./scripts/init-fpga.sh; ./patches/apply.sh	Lesson 1 from section 2
B — firmware only	run and study the protocols on boards that already hold the EDHOC bitstream	cd fpga/sw/export; toolchain + OpenOCD (Lesson 2, section 1)	Lesson 1, section 6, then Lesson 2

- The repository is **private**: ask the author to add your GitHub account, then clone with your own SSH key.
- No prebuilt bitstream ships with the course: route B needs a bitstream built from the **same revision** (route A), e.g. by your lab, written to the boards' flash or kept as a *.bit*.
- The firmware tree *fpga/sw/export/* is self-contained (all code vendored) and needs only the toolchain, OpenOCD and Python — not Chipyard.

1. Start Here (6/8) Readiness check

```
Run this — from the repository root
vivado -version | head -1                       # Vivado v2024.2 ...
riscv64-unknown-elf-gcc --version | head -1     # ... 15.1.0
openocd --version 2>&1 | head -1                # Open On-Chip Debugger 0.12.0 ...
cd fpga/sw/export && ./common/board/list_boards.sh
```

```
Example output — our lab (your serials differ)
C232HM JTAG   FTA6GPSN  (C232HM-DDHSL-0)
Arty console  210319B4A439  /dev/ttyUSB4
C232HM JTAG   FTA6JVAK  (C232HM-DDHSL-0)
Arty console  210319B269F6  /dev/ttyUSB1
```

- Expect **two** Arty lines, each with a tty, and **two** C232HM lines.

If	Check first
an Arty is missing	USB cable, board power switch, Vivado cable drivers
an Arty has no tty	dialout / plugdev group (log out and in), Vivado holding the port
a C232HM is missing	USB plug, OpenOCD udev rules (0403:6014)

1. Start Here (7/8) How to read the practical slides

```
Run this — from fpga/sw/export/tests
make gpio_test
```

```
Example output — board A
GPIO TEST: PASS (0 failures)
```

```
Simplified — common/src/sd_card.c
while (busy) ...   /* shortened: not runnable */
```

- **Run this** boxes are complete commands, in order, with the directory they start from. **Example output** boxes are real console text. **Simplified** boxes shorten source code with "..." — read them, do not paste them.
- Every lab follows the same steps: **objective → starting state → commands → expected output → what it proves → if it fails**.
- Serial numbers, tty names and times in examples come from our lab; yours differ. Lesson 2 shows how to set up your own.

1. Start Here (8/8) Words used in this course

Term	Meaning	Term	Meaning
EDHOC	Ephemeral Diffie-Hellman Over COSE (RFC 9528)	AEAD	encryption + integrity tag (AES-CCM, AES-GCM, ChaCha20/Poly1305)
initiator / responder	board A / board B: who sends message_1 / who answers	KEM	key encapsulation: (ct, ss) = Encaps(pk); ss = Decaps(sk, ct)
suite	set of algorithms (curve or KEM, hash, AEAD)	ML-KEM	the NIST post-quantum KEM (FIPS 203)
method	how each side proves itself: signature or static DH	AuthKEM	post-quantum EDHOC, method 5 (IETF draft)
CBOR / COSE	compact binary encoding / its security formats	PRK_out	the shared secret EDHOC ends with
CRED / ID_CRED / kid	a public-key credential / a short reference to it / key id	TH_n	transcript hash: everything sent so far
PMOD	Digilent 12-pin expansion socket	PERF / STACK	console lines with timing / stack use

Every later lesson uses these words without repeating them; come back here when one is unclear.

Outline

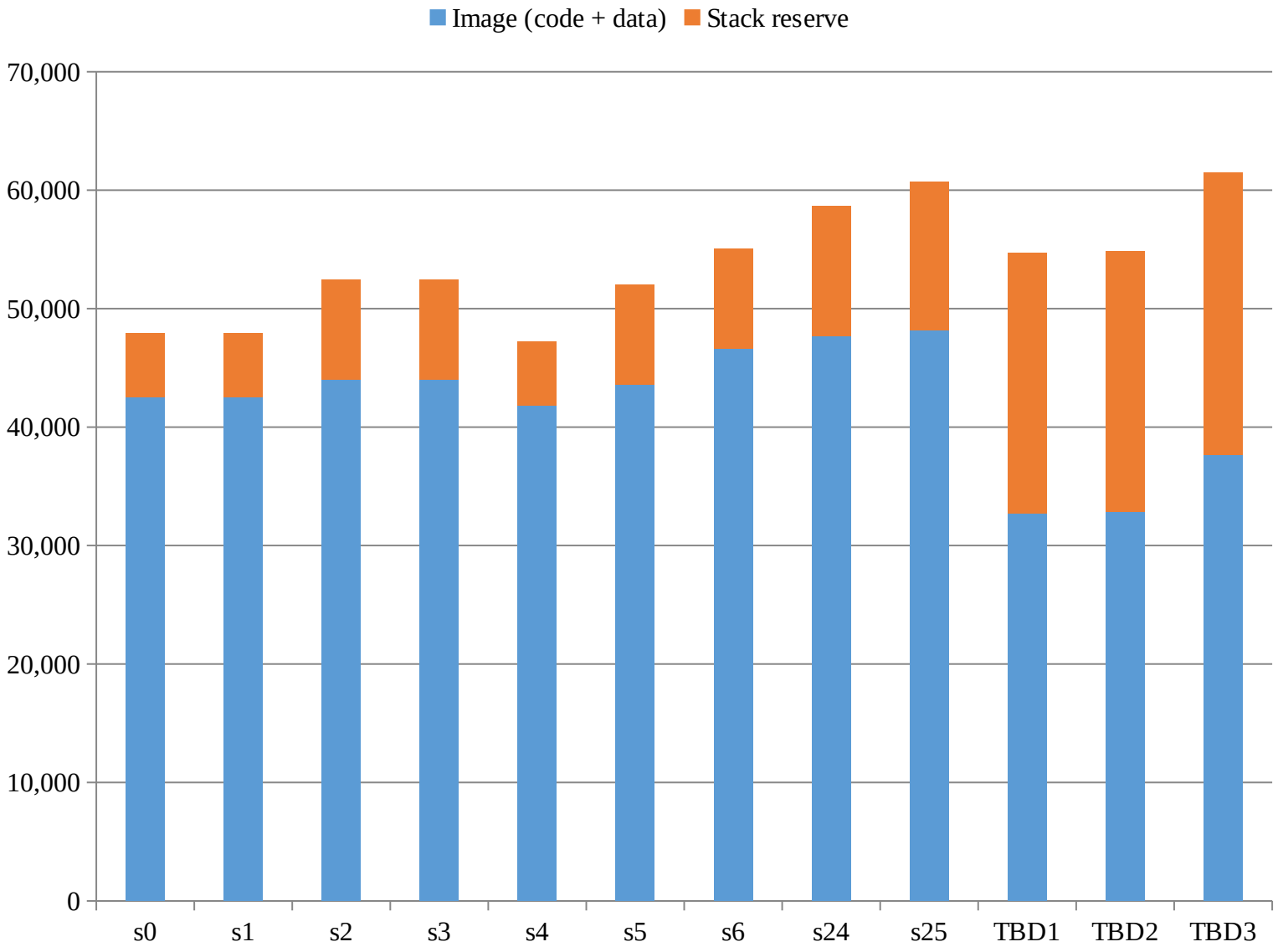
1. Start Here
2. **The System You Will Build**
3. SoC Configuration
4. FPGA Pins & Harness
5. Build & Timing Closure
6. Programming & Bring-up
7. Practice

2. The System You Will Build (1/4) The SoC you will build

Item	Setting	Where it is set
Core	one small Rocket, RV64IMAC, no FPU	EdhocRocket64Config: WithNSmallCores(1)
L1 caches	8 KiB I$ + 8 KiB D$	WithL1ICacheSets / WithL1DCacheSets(128)
RAM	64 KiB on-chip scratchpad at 0x8000_0000, no DDR	WithMbusScratchpad(64 << 10), WithNoMemPort
Clock	75 MHz for every bus	WithTinyArty100TTweaks(75)
Peripherals	8 GPIO, 2 UART, 3 SPI, JTAG debug	WithDefaultPeripherals
Pins	SD on JA, UART1 on JB, radio on JC, JTAG on JD	WithArtyJCRadio + shell overlays

- Section 3 shows each setting in the code, section 4 the pins, section 5 the build, section 6 how to check every peripheral on the board.

2. The System You Will Build (2/4) How much of the 64 KiB each build uses



- Bytes per cipher suite: the image and the stack reserve of its largest build (EDHOC suites 0–25 in Lesson 3, post-quantum TBD1–3 in Lesson 4).
- Every build of the course fits **64 KiB = 65,536 B**; the largest are suite 25 (60,753 B) and TBD3 (61,471 B).
- Check any build yourself: Lesson 2, “Will my build fit?”.

Default flags: TRANSPORT=rf; no TIMING, DEBUG or APP options.

2. The System You Will Build (3/4) The 75 MHz clock: where it is set, how to check it

```
Hardware
new WithTinyArty100TTweaks(75) ++   // every bus clock, in MHz (Configs.scala)
```

```
Firmware
PERF_CPU_HZ / F_CPU = 75000000      // UART, SPI and timing math (firmware)
```

To change the clock	Then expect
1. Change the number in Configs.scala and in the firmware	—
2. Build the bitstream (section 5)	about 1 h
3. Read report/timing.txt	WNS ≥ 0 and 0 failing endpoints, or the bitstream is not usable
4. Program, then run the bring-up tests (section 6)	every test PASS

75 MHz meets timing in this flow (Vivado 2024.2). Exercise 3 asks you to try a faster clock.

2. The System You Will Build (4/4) The two-board system

Board A — Initiator
Arty A7-100T

Board B — Responder
Arty A7-100T

Rocket RV64 · 75 MHz
64 KiB scratchpad · 8 KiB I$/D$

2.4 GHz IEEE 802.15.4 radio
EDHOC handshake + encrypted file

Rocket RV64 · 75 MHz
64 KiB scratchpad · 8 KiB I$/D$

JA
microSD

JB
UART1

JC
MRF24J40

JD
JTAG

JA
microSD

JB
UART1

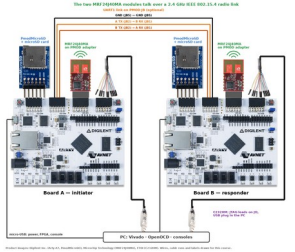
JC
MRF24J40

JD
JTAG

PC: OpenOCD + C232HM
USB-UART console

PC: OpenOCD + C232HM
USB-UART console

- Same bitstream on both boards; the firmware decides the role.
- SD card on JA holds SEND.BIN (board A) and RECV.BIN (board B).



Full-size picture: section 1, (3/8).

Outline

1. Start Here
2. The System You Will Build
3. **SoC Configuration**
4. FPGA Pins & Harness
5. Build & Timing Closure
6. Programming & Bring-up
7. Practice

3. SoC Configuration (1/7) One config class builds the SoC

The bitstream is built from **EdhocRocket6475MHzArty100TConfig** in *fpga/src/main/scala/arty100t/Configs.scala*:

```
class EdhocRocket6475MHzArty100TConfig extends Config(
  new WithArtyJCRadio ++                    // board-to-board link: MRF24J40 on PMOD JC (SPI1)
  new WithTinyArty100TTweaks(75) ++         // Arty shell, peripherals, all bus clocks = 75 MHz
  new chipyard.config.WithBroadcastManager ++   // coherence without an L2 cache
  new chipyard.EdhocRocket64Config          // the SoC itself (next slide)
)
```

- Chipyard configs are a chain of fragments joined with ++; a fragment on the **left overrides** the ones on its right.
- Read it bottom-up: the SoC → how memory coherence works → the board shell and clock → the radio pin variant.
- Same pattern as the Chipyard lecture (04: *MyRocketConfig* on Arty 35T), on the 100T board.

3. SoC Configuration (2/7) EdhocRocket64Config: the SoC

generators/chipyard/src/main/scala/config/EdhocConfigs.scala

```
class EdhocRocket64Config extends Config(
  new freechips.rocketchip.subsystem.WithL1ICacheSets(128) ++     // 8 KiB I-cache
    new freechips.rocketchip.subsystem.WithL1DCacheSets(128) ++   // 8 KiB D-cache
    new chipyard.harness.WithDontTouchChipTopPorts(false) ++
    new testchipip.soc.WithMbusScratchpad(base = 0x80000000L,
                                          size = BigInt(64 << 10)) ++  // 64 KiB on-chip SRAM
    new freechips.rocketchip.subsystem.WithNoMemPort ++           // no DDR / off-chip memory
    new freechips.rocketchip.subsystem.WithNSmallCores(1) ++      // one small Rocket, RV64
    new freechips.rocketchip.subsystem.WithoutTLMonitors ++
    new chipyard.config.AbstractConfig)
```

Fragment	Effect
WithNSmallCores(1)	1 Rocket tile, RV64IMAC, no FPU
WithL1I/DCacheSets(128)	128 sets → 8 KiB caches
WithMbusScratchpad	64 KiB SRAM at 0x8000_0000 (BRAM)
WithNoMemPort	no DDR: all code + data on chip
WithoutTLMonitors	smaller, faster (no protocol checkers)

- Everything the firmware uses — code, data, stack — lives in the 64 KiB scratchpad. There is no external RAM on this SoC.
- The Arty’s 256 MB DDR3 is left unused: the requirement asks for on-chip RAM, and the whole firmware fits on chip.

3. SoC Configuration (3/7) Cache size and fragment order

128 sets × **1 way** × **64 B / line** = **8 KiB per cache**

- `WithNSmallCores(1)` creates the tile with small default caches; `WithL1ICacheSets` / `WithL1DCacheSets` **rewrite** those tile parameters.
- A rewrite only sees tiles that already exist, so the cache fragments must stay **to the left** of `WithNSmallCores` in the chain.
- Wrong order builds without error but gives the default cache size — check the generated *.dts* / *.json* or the utilization report after a change.
- Small core = RV64IMAC: integer, multiply/divide, atomics, compressed; no floating point (the firmware is integer-only).

3. SoC Configuration (4/7) Memory map

Address	Size	Device	Used for
0x0000_0000	4 KiB	Debug module	JTAG debugging / program loading
0x0001_0000	8 KiB	Boot ROM (sdboot)	copies the firmware from the SD card at reset
0x0200_0000	64 KiB	CLINT	timer, software interrupts
0x0C00_0000	64 MiB	PLIC	external interrupts (unused by the firmware)
0x6400_0000	4 KiB	UART0	USB-UART console, 57600 baud
0x6400_1000	4 KiB	SPI0	microSD card (PMOD JA)
0x6400_2000	4 KiB	GPIO, 8 pins	4 LEDs, RGB LED, radio ~RST / INT
0x6400_3000	4 KiB	UART1	PMOD JB (optional wired link)
0x6400_5000	4 KiB	SPI2	spare
0x6400_6000	4 KiB	SPI1	MRF24J40 radio (PMOD JC)
0x8000_0000	64 KiB	Scratchpad SRAM	code, data and stack

The firmware's *common/include/platform.h* uses the same addresses.

3. SoC Configuration (5/7) Peripherals and their addresses

fpga/src/main/scala/arty100t/Configs.scala

```
class WithDefaultPeripherals extends Config((site, here, up) => {
  case PeripheryUARTKey => List(
    UARTParams(address = BigInt(0x64000000L)),  // UART0 - USB UART
    UARTParams(address = BigInt(0x64003000L)))  // UART1 - PMOD JB (with the radio)
  case PeripherySPIKey => List(
    SPIParams(rAddress = BigInt(0x64001000L)),  // SPI0 - SD card
    SPIParams(rAddress = BigInt(0x64006000L)),  // SPI1 - radio
    SPIParams(rAddress = BigInt(0x64005000L)))  // SPI2
  case PeripheryGPIOKey => List(GPIOParams(address = BigInt(0x64002000L), width = 8))
})
```

- The UART, SPI and GPIO blocks are SiFive's open-source peripherals (*sifive-blocks*), the same as in the Chipyard lecture (04).
- Each list entry is one controller instance; the list index is the port number used by the overlays in section 4 (SPI0 → JA, SPI1 → JC).
- Addresses are free to choose inside the peripheral bus window; keep them in step with the firmware headers.

3. SoC Configuration (6/7) One clock for the whole SoC

fpga/src/main/scala/arty100t/Configs.scala

```
class WithTinyArty100TTweaks(freqMHz: Double = 50) extends Config(
  new chipyard.harness.WithHarnessBinderClockFreqMHz(freqMHz) ++
  new chipyard.config.WithMemoryBusFrequency(freqMHz) ++
  new chipyard.config.WithFrontBusFrequency(freqMHz) ++
  new chipyard.config.WithSystemBusFrequency(freqMHz) ++
  new chipyard.config.WithPeripheryBusFrequency(freqMHz) ++
  new chipyard.config.WithControlBusFrequency(freqMHz) ++
  new chipyard.config.WithOffchipBusFrequency(freqMHz) ++
  new chipyard.clocking.WithPassthroughClockGenerator ++
  new WithArty100TUARTHarnessBinder ++ new WithArty100TSPISDCardHarnessBinder ++
  new WithArty100TJTAGHarnessBinder ++ new WithArty100TGPIOHarnessBinder ++
  ...
  new WithDefaultPeripherals ++ new WithSystemModifications ++ ...)
```

- `(75)` sets all seven bus clocks at once.
- Chipyard 1.11 added front / control / off-chip bus clocks that default to 500 MHz; every bus in the clock group must ask for the same frequency.
- The board's 100 MHz oscillator (pin E3) feeds a PLL that makes 75 MHz.

Firmware must know the clock too: `PERF_CPU_HZ` / `F_CPU` = 75000000 (UART divider, SPI dividers, timing reports).

3. SoC Configuration (7/7) Boot ROM rebuilt for RV64

```
fpga/src/main/scala/arty100t/Configs.scala
class WithSystemModifications extends Config((site, here, up) => {
  case BootROMLocated(x) => up(BootROMLocated(x), site).map{ p =>
    val freqMHz = (site(SystemBusKey).dtsFrequency.get / (1000 * 1000)).toLong
    val make = s"make -C fpga/src/main/resources/arty35t/sdboot " +
               s"XLEN=${site(XLen)} PBUS_CLK=${freqMHz}"
    require (make.! == 0, "Failed to build bootrom")
    p.copy(hang = 0x10000, size = 0x2000,
           contentFileName = s"./fpga/src/main/resources/arty35t/sdboot/build/sdboot.bin")
  }
  ...
```

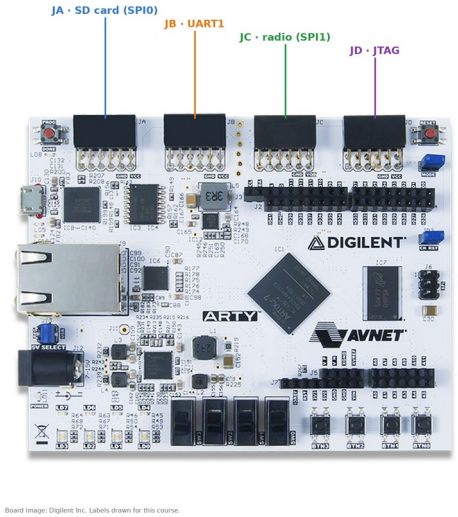
- Every bitstream build recompiles the boot ROM for the core width (`XLEN=64`) and the bus clock (`PBUS_CLK=75`).
- At reset the boot ROM copies the firmware from the SD card (LBA 2048) into the scratchpad and jumps to 0x8000_0000.
- During development you can skip it: firmware is loaded over JTAG in seconds (section 6).

Outline

1. Start Here
2. The System You Will Build
3. SoC Configuration
4. **FPGA Pins & Harness**
5. Build & Timing Closure
6. Programming & Bring-up
7. Practice

4. FPGA Pins & Harness (1/9) PMOD map

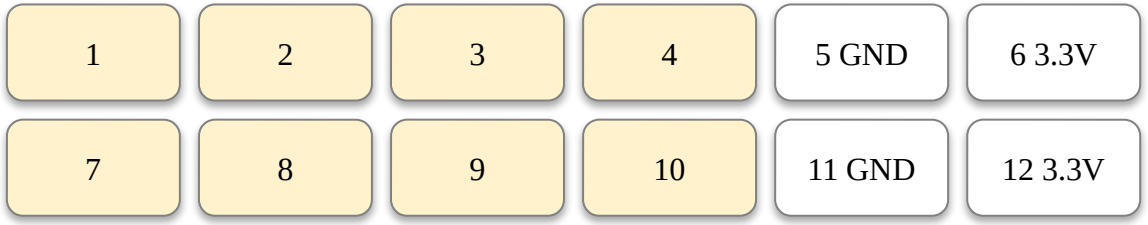
PMOD	Module	Controller	Pins (PMOD pin: FPGA pin)
JA	PmodMicroSD	SPI0 @0x6400_1000	1 CS: G13 · 2 MOSI: B11 · 3 MISO: A11 · 4 SCK: D12
JB	UART1	UART1 @0x6400_3000	1 RX: E15 · 2 TX: E16
JC	MRF24J40 radio	SPI1 @0x6400_6000 + GPIO 5 / 7	1 CS: U12 · 2 MOSI: V12 · 3 MISO: V10 · 4 SCK: V11 · 7 INT: U14 · 8 ~RST: V14
JD	JTAG (C232HM cable)	Debug module	1 TDI: D4 · 2 TMS: D3 · 3 TCK: F4 · 4 TDO: F3



Board image: Digilent Inc. Labels drawn for this course.

- Same wiring on both boards.
- `WithArtyJCRadio` selects this map (radio on JC, UART1 on JB).
- The USB-UART console (UART0) uses the on-board FTDI chip, no PMOD.

4. FPGA Pins & Harness (2/9) Wiring: pin numbers, direction, power

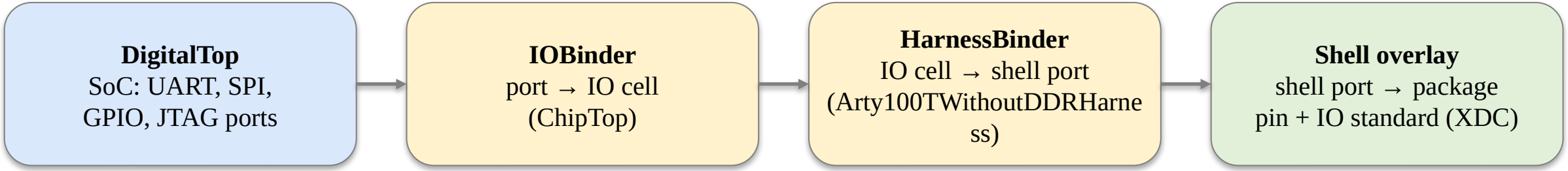


- Every PMOD is a 2 × 6 socket: top row 1–6, bottom row 7–12. Pins 5 / 11 are GND, 6 / 12 are 3.3 V. Pin 1 is marked on the board's silkscreen.

PMOD	FPGA → module (outputs)	module → FPGA (inputs)	Power
JA SD	1 CS, 2 MOSI, 4 SCK	3 MISO	5 GND, 6 3.3 V
JB UART1	2 TX	1 RX	5 GND; to the other board: TX → RX, RX ← TX, GND
JC radio	1 CS, 2 MOSI, 4 SCK, 8 ~RST	3 MISO, 7 INT	5 / 11 GND, 6 / 12 3.3 V
JD JTAG	4 TDO	1 TDI, 2 TMS, 3 TCK	5 GND (black wire); red VCC unconnected

- Same wiring on both boards. The SD and radio modules take 3.3 V from the PMOD; the C232HM only shares ground.
- Check before power-on: every module's GND on pin 5 (and 11), no 5 V anywhere.

4. FPGA Pins & Harness (3/9) From SoC port to FPGA pin



File (fpga/src/main/scala/arty100t/)	What this tree changes
Configs.scala	EDHOC configs, peripherals, clock tweak, radio switch
IOBinders.scala	SPI / GPIO punch-through ports (WithArtySPIIOPunchthrough)
HarnessBinders.scala	UART, SPI-SD, JTAG (PMOD JD), GPIO binders
CustomShell.scala	shell with 3 SPI overlays, 2 UARTs, JTAG, GPIO
CustomOverlay.scala	package pins of every overlay (next slides)

Same three levels as in the Chipyard lecture (04): DigitalTop → ChipTop → ChipHarness.

4. FPGA Pins & Harness (4/9) SD card on PMOD JA (SPI0)

```
CustomOverlay.scala — SDIOArtyPlacedOverlay
case 0 => Seq(("D12", IOPin(io.spi_clk)),    // SPI0 - PMOD JA (SD card)
              ("B11", IOPin(io.spi_cs)),
              ("A11", IOPin(io.spi_dat(0))),
              ("D13", IOPin(io.spi_dat(1))),
              ("B18", IOPin(io.spi_dat(2))),
              ("G13", IOPin(io.spi_dat(3))))
```

Overlay pin	Carries	PMOD pin	PmodMicroSD
G13 spi_dat(3)	chip select	JA1	CS
B11 spi_cs	MOSI (command)	JA2	MOSI
A11 spi_dat(0)	MISO (data 0)	JA3	MISO
D12 spi_clk	clock	JA4	SCK



PmodMicroSD (pin 1 = CS, top); image: Digilent Inc.

- The SDIO overlay uses **SD-card pin names**: the pin called *spi_cs* carries MOSI, and CS goes out on *dat(3)*.
- Pull-ups on CS, MOSI and MISO.
- Same pins as upstream Chipyard's Arty 100T SD slot.
- Card init at ~150 kHz, data at ~2.3 MHz (SPI mode).

4. FPGA Pins & Harness (5/9) MRF24J40 radio on PMOD JC (SPI1 + GPIO)

```
CustomOverlay.scala — SPI1 with the radio
case 1 if shell.p(ArtyJCRadioKey) =>
  Seq(("V11", IOPin(io.spi_clk)),    // JC4 SCK
      ("V12", IOPin(io.spi_cs)),     // JC2 MOSI
      ("V10", IOPin(io.spi_dat(0))), // JC3 MISO
      ("F6",  IOPin(io.spi_dat(1))), // unused (freed RGB LED pin)
      ("G4",  IOPin(io.spi_dat(2))), // unused (freed RGB LED pin)
      ("U12", IOPin(io.spi_dat(3)))) // JC1 CS
```

```
CustomOverlay.scala — GPIO
val gpioLocations = List("H5", "J5", "T9", "T10",  // 4 LEDs
                         "E1",                     // RGB LED
                         if (radio) "V14" else "F6", // JC8 ~RST
                         "G6",
                         if (radio) "U14" else "G4") // JC7 INT
```



MRF24J40MA, on a PMOD adapter on JC; image: Microchip Technology.

- `WithArtyJCRadio` sets `ArtyJCRadioKey = true`; the overlays then pick the radio pinout.
- MRF24J40MA: IEEE 802.15.4, 2.4 GHz, 250 kbit/s; SPI at 4.7 MHz.
- GPIO 5 drives the radio's reset, GPIO 7 reads its interrupt line (polled).
- Two RGB-LED pins are given up for this.

4. FPGA Pins & Harness (6/9) Radio adapter: module pins to PMOD JC

MRF24J40MA pin	Signal	Direction	PMOD JC pin	FPGA pin
8 CS	chip select	FPGA → radio	JC1	U12
5 SDI	SPI data in (MOSI)	FPGA → radio	JC2	V12
7 SDO	SPI data out (MISO)	radio → FPGA	JC3	V10
6 SCK	SPI clock	FPGA → radio	JC4	V11
4 INT	interrupt (polled by GPIO 7)	radio → FPGA	JC7	U14
2 RESET	reset, active low (GPIO 5)	FPGA → radio	JC8	V14
10 VIN	3.3 V supply	—	JC6 (or 12)	—
1, 11, 12 GND	ground	—	JC5 (or 11)	—
3 WAKE, 9 NC	not used by the firmware	—	—	—

- Connect the module signals to the JC pins shown here; check the module pin numbers against the MRF24J40MA data sheet.
- The JC column is fixed by the bitstream (section 4, (5/9)).

4. FPGA Pins & Harness (7/9) UART0 console and UART1 on PMOD JB

```
CustomOverlay.scala — UARTArtyPlacedOverlay
// UART0: USB UART (A9=RX, D10=TX)
// UART1: PMOD C (U12=RX, V12=TX), or PMOD B (E15=RX, E16=TX) with the radio
val packagePinsWithPackageIOs = shellInput.index match {
  case 0 => Seq(("A9",  IOPin(io.rxd)),    // USB UART RX
                ("D10", IOPin(io.txd)))    // USB UART TX
  case 1 if shell.p(ArtyJCRadioKey) => Seq(
                ("E15", IOPin(io.rxd)),    // PMOD B JB1 - RX
                ("E16", IOPin(io.txd)))    // PMOD B JB2 - TX
  case 1 => Seq(("U12", IOPin(io.rxd)),    // PMOD C JC1 - RX
                ("V12", IOPin(io.txd)))    // PMOD C JC2 - TX
}
```

- UART0 = console through the board's USB port, 57600 baud, 8N1.
- UART1 = optional wired link between the boards (TRANSPORT=uart).
- With the radio, UART1 moves from JC to JB.

- Each board shows up as two ttyUSB devices; the console is interface 1 of the Digilent FTDI. Find it by serial number, because ttyUSB numbers change when Vivado or OpenOCD opens a port (section 6).

4. FPGA Pins & Harness (8/9) JTAG on PMOD JD with a C232HM cable

C232HM wire	FTDI pin	Signal	PMOD JD	FPGA pin
Orange	ADBUS0	TCK	JD3	F4
Yellow	ADBUS1	TDI	JD1	D4
Green	ADBUS2	TDO	JD4	F3
Brown	ADBUS3	TMS	JD2	D3
Black	GND	GND	JD5 (GND)	–

- The core's debug module is wired to PMOD JD, not to the board's USB JTAG (that one programs the FPGA).
- One C232HM per board; scripts pick the cable by its FTDI serial (*common/board/boards.env*).
- JTAG clock 1 MHz (`adapter speed 1000`).

`fpga/sw/export/common/board/c232hm.cfg`

```
source [find interface/ftdi/c232hm.cfg]
transport select jtag
adapter speed 1000
# RISC-V debug DTM: 5-bit IR
jtag newtap riscv tap -irlen 5 -ignore-version
target create riscv.cpu riscv -chain-position riscv.tap
```



C232HM-DDHSL-0: flying leads to JD, USB to the PC; image: FTDI.

4. FPGA Pins & Harness (9/9) Makefile entry

```
fpga/Makefile
ifeq ($(SUB_PROJECT),arty100t)
        SBT_PROJECT       ?= fpga_platforms
        MODEL             ?= Arty100TWithoutDDRHarness   # was Arty100THarness
        VLOG_MODEL        ?= Arty100TWithoutDDRHarness
        MODEL_PACKAGE     ?= chipyard.fpga.arty100t
        CONFIG            ?= SmallRocket32Arty100TConfig  # default; override on the command line
        CONFIG_PACKAGE    ?= chipyard.fpga.arty100t
        GENERATOR_PACKAGE ?= chipyard
        ...
```

- `SUB_PROJECT ?= arty100t` is now the default sub-project (upstream: vcu118).
- **Arty100TWithoutDDRHarness**: the harness without the DDR3 controller (the SoC has no memory port).
- The EDHOC config is chosen at build time: `CONFIG=EdhocRocket6475MHzArty100TConfig` (next section).

Outline

1. Start Here
2. The System You Will Build
3. SoC Configuration
4. FPGA Pins & Harness
5. **Build & Timing Closure**
6. Programming & Bring-up
7. Practice

5. Build & Timing Closure (1/5) Build the bitstream

```
Run this — about 1 h on a recent PC
cd chipyard                # repository root, route A set up (section 1)
source env.sh
cd fpga
make SUB_PROJECT=arty100t CONFIG=EdhocRocket6475MHzArty100TConfig bitstream
echo $?                    # 0 = make finished; nonzero = something failed: read the log
```

- Output folder: *fpga/generated-src/chipyard.fpga.arty100t.Arty100TWithoutDDRHarness.EdhocRocket6475MHzArty100TConfig/obj/*
  - *Arty100TWithoutDDRHarness.bit* — the bitstream
  - *report/timing.txt*, *report/utilization.txt* — check these after every build
  - *post_opt.dcp*, *post_route.dcp* — checkpoints for timing closure
- **Careful:** exit status 0 does not mean timing was met, and a failed flow may still leave a *.bit*. Timing is decided only by *report/timing.txt*: read it before using a new bitstream.

5. Build & Timing Closure (2/5) Resources used

Resource	Used	Available (XC7A100T)	Use
Slice LUTs	14,767	63,400	23.3 %
Flip-flops	6,716	126,800	5.3 %
Block RAM (36 Kb)	34 + 10 half	135	28.9 %
DSP slices	10	240	4.2 %

- 16 of the 34 block RAMs are the 64 KiB scratchpad (16 × 4 KiB).
- The rest: caches, boot ROM, peripheral FIFOs.
- Your numbers should be close to these; the scratchpad must use 16 RAMB36.

From *report/utilization.txt* (`report_utilization -hierarchical`):

```
| Instance                  | Module          | Total LUTs |  FFs | RAMB36 | RAMB18 | DSP |
| Arty100TWithoutDDRHarness |           (top) |      14767 | 6716 |     34 |     10 |  10 |
|   chiptop0                | ChipTop         |      14761 | 6630 |     34 |     10 |  10 |
|     system                | DigitalTop      |      14759 | 6624 |     34 |     10 |  10 |
|       bank                | ScratchpadBank |         67 |  109 |     16 |      0 |   0 |
```

5. Build & Timing Closure (3/5) Reading the timing report

report/timing.txt — Design Timing Summary, 75 MHz build:

```
    WNS(ns)  TNS(ns)  TNS Failing Endpoints  TNS Total Endpoints   WHS(ns)  THS(ns)  THS Failing Endpoints
      0.002    0.000                      0                24148     0.037    0.000                      0
Clock                      WNS(ns)  TNS(ns)  TNS Failing  TNS Total   WHS(ns)
  clk_out1_harnessSysPLL     0.002    0.000            0      23458     0.037
JTCK                        41.450    0.000            0        548     0.109
```

- **WNS** (worst negative slack) ≥ 0 and **0 failing endpoints** for setup (TNS) and hold (THS) = timing met.
- *clk_out1_harnessSysPLL* is the 75 MHz system clock; *JTCK* is the JTAG clock from PMOD JD (10 MHz constraint, easily met).

5. Build & Timing Closure (4/5) If WNS is slightly negative: the closure recipe

- 75 MHz has little margin in this design: a change of pins or flow can move WNS a little below 0 (e.g. −0.17 ns). Then do not rebuild from scratch.
- Run this in Vivado (Tcl console), from the *obj/* folder of the build:

```
open_checkpoint post_opt.dcp
place_design    -directive Explore
phys_opt_design -directive AlternateFlowWithRetiming
route_design    -directive AggressiveExplore
phys_opt_design -directive Explore
route_design    -directive AggressiveExplore -tns_cleanup
phys_opt_design -directive Explore
file mkdir report
report_timing_summary -file report/timing.txt ;# check WNS and WHS >= 0
write_bitstream Arty100TWithoutDDRHarness.bit
```

- Expect setup and hold met (WNS, WHS ≥ 0); then accept the bitstream as in (5/5).
- Extra clock uncertainty did not improve the measured implementation; do not assume it will repair a timing failure.
- Vivado 2024.2 *phys_opt_design* sometimes crashes after routing (exit 139): run each attempt as its own process from a checkpoint.

5. Build & Timing Closure (5/5) How to accept a new bitstream

Step	Look at	Accept when
1. Build	exit status of make, then the log	0; nonzero = build failed
2. Timing	Design Timing Summary in report/timing.txt	WNS ≥ 0 and WHS ≥ 0, 0 failing endpoints (TNS and THS)
3. Area	report/utilization.txt	ScratchpadBank uses 16 RAMB36 (64 KiB)
4. Program	program_two.tcl (section 6)	PROGRAMMED <serial> for both boards
5. Bring-up	jtag, gpio, spi, uart tests (section 6)	each ends in PASS; jtag_test shows MXL = 2, ACIMX
6. Keep it	copy the .bit with config name and date	—

Run this — step 2

```
cd fpga/generated-src/chipyard.fpga.arty100t.Arty100TWithoutDDRHarness.EdhocRocket6475MHzArty100TConfig/obj
grep -A6 "Design Timing Summary" report/timing.txt
```

Do not skip step 2 because the board seems to work: a design that misses timing can fail only now and then, in ways that look like software bugs.

Outline

1. Start Here
2. The System You Will Build
3. SoC Configuration
4. FPGA Pins & Harness
5. Build & Timing Closure
6. **Programming & Bring-up**
7. Practice

6. Programming & Bring-up (1/5) Program both FPGAs

```
Run this — SERIALS are our boards: use your two Arty serials (list_boards.sh)
cd fpga/sw/export
BIT=../../generated-src/chipyard.fpga.arty100t.Arty100TWithoutDDRHarness.\
EdhocRocket6475MHzArty100TConfig/obj/Arty100TWithoutDDRHarness.bit \
SERIALS="210319B269F6 210319B4A439" \
vivado -mode batch -source common/board/program_two.tcl
```

```
PROGRAMMED 210319B269F6
PROGRAMMED 210319B4A439
```

- SERIALS = the Digilent USB serial of each Arty, as printed by *common/board/list_boards.sh*. The script uses a running Vivado GUI's hardware server if there is one.
- At power-up the FPGA loads whatever its QSPI flash holds (e.g. an older design). Symptom: OpenOCD reports "*dtmcontrol is 0*" → program again.
- For a board that starts the EDHOC SoC by itself, write the bitstream to the QSPI flash (Vivado → Add Configuration Memory Device).

6. Programming & Bring-up (2/5) Load firmware over JTAG

```
Run this — from the repository root
cd fpga/sw/export/tests
make gpio_test                              # build a test program
BOARD=initiator ../common/board/run_one.sh build/gpio_test.elf 5   # board A
BOARD=responder ../common/board/run_one.sh build/gpio_test.elf 5   # board B
```

```
Simplified — what run_one.sh does ($SERIAL, $ELF are set by the script)
openocd -c "adapter serial $SERIAL" -f c232hm.cfg \
  -c "halt" -c "load_image $ELF" \
  -c "resume 0x80000000" -c "shutdown"
```

```
Run this — which tty is which board?
../common/board/list_boards.sh     # serial and tty of every board and cable
```

- `run_one.sh`: halt the core, write the ELF into the scratchpad, start at 0x8000_0000, show the console.
- Both boards at once (responder first): `load_both.sh <initiator.elf> <responder.elf>`, used from Lesson 3 on.
- Cable and console of each board come from *boards.env*; Lesson 2 shows how to fill it in with your serials.
- The console tty is found by the board's serial, so moving ttyUSB numbers do not matter (57600 baud).

6. Programming & Bring-up (3/5) Bring-up: GPIO and JTAG

```
tests/build/gpio_test.elf
=== GPIO test (0x64002000, 8 pins) ===
  output_en   : got=ff want=ff PASS
  all low     : got=0 want=0 PASS
  all high    : got=ff want=ff PASS
  walking ones (watch the LEDs):
    walk: PASS
  xor set     : got=ff want=ff PASS
  pattern AA  : got=aa want=aa PASS
  pattern 55  : got=55 want=55 PASS
GPIO TEST: PASS (0 failures)
```

```
tests/build/jtag_test.elf
=== JTAG / debug module test ===
  Loaded and resumed over JTAG: PASS (this output is the proof)
  mhartid     : 0
  mvendorid   : 0
  marchid     : 1
  mimpid      : 20181004
  misa low32  : 801105  (MXL=2 -> RV64)
  extensions  : ACIMX
JTAG TEST: PASS (0 failures)
```

- Real output from board A. The GPIO walk lights the four LEDs one after another.
- *misa* confirms the hardware: MXL = 2 → 64-bit; extensions A, C, I, M (X = non-standard extensions present); *marchid* 1 = Rocket.

6. Programming & Bring-up (4/5) Bring-up: SPI (SD card) and UART

```
tests/build/spi_test.elf
=== SPI test (SPI0 @ 0x64001000, SD slot) ===
  sckdiv      : got=c want=c PASS
  sckmode     : got=0 want=0 PASS
  csdef       : got=1 want=1 PASS
  SD CMD0     : R1=1 PASS (card in idle state)
SPI TEST: PASS (0 failures)
```

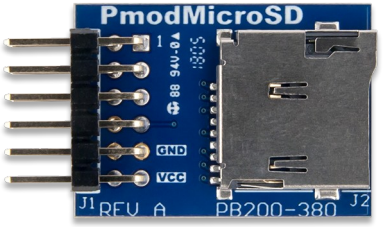
```
tests/build/uart_test.elf
=== UART test ===
  UART0 @ 0x64000000 (console), UART1 @ 0x64003000 (PMOD JB)
  UART0 console TX: PASS (you are reading it)
  UART0 div    : got=2a want=2a PASS
  UART1 div    : got=2a want=2a PASS
  UART1 txen   : got=1 want=1 PASS
  UART1 rxen   : got=1 want=1 PASS
  UART1 loop   : SKIP (no echo; jumper JB1-JB2 to test)
UART TEST: PASS (0 failures)
```

- **SD CMD0 → R1 = 0x01** proves clock, chip select and both data lines work against a real card.
- Right after power-up the card can answer the first CMD0 with one junk byte (e.g. 0x3f). The boot ROM, this test and the firmware send CMD0 again; the second answer is clean.
- UART1 loopback needs a jumper JB1–JB2; without it the step is skipped, not failed.

6. Programming & Bring-up (5/5) Booting from the SD card

```
Run this — from the repository root; sdX = your card reader (lsblk)
cd fpga/sw/export/tests && make gpio_test      # -> build/gpio_test.bin
sudo ../tools/mksd.sh /dev/sdX                    # once; ERASES the card
sudo ../common/board/sd_boot.sh build/gpio_test.bin sdX1    # image -> LBA 2048
```

Card area	Content
LBA 0	MBR partition table
LBA 2048 … (partition 1, raw, 32 MiB)	firmware image read by the boot ROM
LBA 67584 … (partition 2, FAT32)	files: SEND.BIN / RECV.BIN (Lesson 5)



- At reset the boot ROM copies 126 sectors (63 KiB) from LBA 2048 into the scratchpad and jumps there.
- Check: put the card into JA, press RESET (also right after inserting the card). Expect *GPIO TEST: PASS* on the console within a few seconds. The boot ROM's own lines (INIT … BOOT) come first at another baud rate and look garbled at 57600.
- Only garbled lines, no PASS: the boot ROM found no valid image at LBA 2048. Write it again.
- No card reader? First format the card with *sd_prep* (Lesson 5). In *tests/*: `cp build/gpio_test.bin build/boot.bin`, then `make -B sd_prep EXTRA_CFLAGS=-DSD_PREP_BOOT`. Load it with `BOARD=initiator ../common/board/run_one.sh build/sd_prep.elf 30`, then press RESET.

Outline

1. Start Here
2. The System You Will Build
3. SoC Configuration
4. FPGA Pins & Harness
5. Build & Timing Closure
6. Programming & Bring-up
7. **Practice**

7. Practice (1/3) Exercises

Exercise 1:

- Add **EdhocRocket128K75MHzArty100TConfig**: the same SoC with a 128 KiB scratchpad.
- Build it. How many block RAMs does the scratchpad use now? Does 75 MHz still close?

Exercise 2:

- Program both boards with the EDHOC bitstream and run `gpio_test`, `spi_test`, `jtag_test` and `uart_test` on each.
- Put a jumper on JB1–JB2 and make the UART1 loopback PASS.

Exercise 3:

- Copy the config with `WithTinyArty100TTweaks(100)` and build it. Read the Design Timing Summary: would you accept this bitstream?

7. Practice (2/3) What to put in the report

- Exercise 1: your config class, the utilization table (block RAM count) and the Design Timing Summary lines (WNS, failing endpoints).
- Exercise 2: the console output of all four tests on both boards, and a photo of your wiring with the PMOD ports labelled.
- Exercise 3: the Design Timing Summary lines and your decision, with the rule from section 5 (5/5).
- For every step: the command you ran and how long it took on your PC.

7. Practice (3/3) Answer notes

Exercise	What should change	What should stay the same	Why
1. 128 KiB	ScratchpadBank: 16 → 32 RAMB36; size = BigInt(128 << 10) in a copy of EdhocRocket64Config	the core, caches and pins	one RAMB36 holds 4 KiB. Timing may or may not still close: read timing.txt. The firmware still uses 64 KiB until memory.lds changes
2. Tests	UART1 loop: SKIP → PASS with the jumper	every other line PASS	SKIP means “not run”; the jumper lets UART1 hear itself
3. 100 MHz	WNS clearly negative (about −3.2 ns in this flow) and thousands of failing endpoints	the build itself still finishes	reject it: the critical path inside the Rocket core needs about 13 ns here

- Negative case to recognise: a bitstream with WNS < 0 may load and even pass a test — reject it anyway (section 5, (5/5)).

Today's lecture highlight:

- *EdhocRocket6475MHzArty100TConfig* = small RV64 Rocket, 8 KiB caches, 64 KiB scratchpad, no DDR, GPIO / 2 UART / 3 SPI / JTAG.
- Every build of the course fits the 64 KiB scratchpad (largest 61.5 KB with its stack); the clock is set in one place and checked in report/timing.txt.
- Pins: JA SD card, JB UART1, JC MRF24J40 radio, JD JTAG — set in the shell overlays, selected by config keys.
- Accept a bitstream only after its timing report: a failing build still writes a .bit file.
- Program both boards, load over JTAG, prove each peripheral with a test program.

Next lesson:

「 Lesson 2 」 Software environment and tests

THANK YOU

2026/12